lec15

March 10, 2026

1 Lecture 15: Asymptotic Tests — Wald, Score, Generalized LRT

Data 145, Spring 2026: Evidence and Uncertainty Instructors: Ani Adhikari, William Fithian

---

Please run all the code cells below before you start reading.

---

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import norm, poisson, chi2
from scipy.optimize import brentq
import ipywidgets as widgets
from IPython.display import display, clear_output

plt.style.use('fivethirtyeight')
%matplotlib inline

# Color scheme
COLOR_DATA = 'steelblue'
COLOR_APPROX = 'firebrick'

# Reproducibility
rng = np.random.default_rng(145)

print("Setup complete.")
```

```
Setup complete.
```

1.1 Introduction

In Lectures 3–5, we developed the MLE and its asymptotic distribution: $\sqrt{n}(\hat{\theta}_n - \theta_0) \xrightarrow{d} N(0, 1/I(\theta_0))$. In Lecture 14, we saw that tests and confidence intervals are two sides of the same coin.

Today we put these ideas together. We develop **three different test statistics** for testing $H_0 : \theta = \theta_0$ in a one-parameter family — the **Wald test**, the **score test**, and the **generalized likelihood ratio test** (GLRT). All three are asymptotically equivalent, and the reason is a beautiful geometric picture: the log-likelihood is approximately a **parabola** near the MLE, and the three tests simply measure different features of that parabola.

1.2 1. Setup: MLE Asymptotics Recap

1.2.1 What we need from Lectures 3–5

We work with iid data $X_1, \dots, X_n \sim f_\theta$, where $\theta \in \Theta \subseteq \mathbb{R}$ is a one-dimensional parameter. Recall:

- **Log-likelihood**: $\ell_n(\theta; X) = \sum_{i=1}^n \log f_\theta(X_i)$
- **Score**: $S_n(\theta; X) = \ell_n'(\theta; X) = \sum_{i=1}^n \ell_1'(\theta; X_i)$
- **Fisher information**: $I(\theta) = \mathrm{Var}_\theta(\ell_1'(\theta; X_i))$
- **MLE asymptotics**: $\hat{\theta}_n \xrightarrow{P} \theta_0$ and $\sqrt{n}(\hat{\theta}_n - \theta_0) \xrightarrow{d} N(0,\, 1/I(\theta_0))$
- **Score CLT**: $\frac{S_n(\theta_0; X)}{\sqrt{nI(\theta_0)}} \xrightarrow{d} N(0, 1)$

1.2.2 The quadratic approximation idea

The key to all three tests is a **Taylor expansion** of the log-likelihood around the MLE $\hat{\theta}_n$. Since $\ell_n'(\hat{\theta}_n) = 0$ (first-order condition for the MLE), the second-order expansion is:

$$\ell_n(\theta; X) \approx \ell_n(\hat{\theta}_n; X) + \frac{1}{2}\ell_n''(\hat{\theta}_n; X)(\theta - \hat{\theta}_n)^2$$

This is a **downward-opening parabola** centered at $\hat{\theta}_n$, with curvature determined by $\ell_n''(\hat{\theta}_n)$. The three tests we'll develop today each measure a different geometric feature of this parabola.

1.2.3 Running example: Poisson

We'll use the Pois(λ) model as a running example throughout the lecture.

Suppose $X_1, \dots, X_n \overset{\text{iid}}{\sim} \text{Pois}(\lambda)$. We want to test $H_0 : \lambda = \lambda_0$.

Quantity	Formula
Log-likelihood	$\ell_n(\lambda; X) = (\sum_i X_i) \log \lambda - n\lambda + C$
Score	$\ell_n'(\lambda; X) = \frac{n(\bar{X} - \lambda)}{\lambda}$
Fisher information	$I(\lambda) = 1/\lambda$
MLE	$\hat{\lambda}_n = \bar{X}$

[2]:
```
# Simulate Poisson data for our running example
n = 30
lam_true = 3.0
x = rng.poisson(lam_true, n)
```

```python
x_bar = x.mean()
s2 = np.var(x, ddof=1)  # sample variance

print(f"n = {n}")
print(f"x̄ (MLE) = {x_bar:.4f}")
print(f"Sample variance = {s2:.4f}")
print(f"\nFor a correctly specified Poisson, Var(X) = E[X] =  ,")
print(f"so we'd expect the sample variance  x̄ = {x_bar:.4f}.")
print(f"Ratio s²/x̄ = {s2/x_bar:.4f}  (  1 if Poisson is correct)")
```

```
n = 30
x̄ (MLE) = 3.0667
Sample variance = 3.5126

For a correctly specified Poisson, Var(X) = E[X] =  ,
so we'd expect the sample variance  x̄ = 3.0667.
Ratio s²/x̄ = 1.1454  (  1 if Poisson is correct)
```

1.3 2. The Wald Test

1.3.1 Idea

We've already used this implicitly! The MLE is approximately normal:

$$\hat{\theta}_n \approx N(\theta_0,\, 1/(nI(\theta_0)))$$

So a natural test rejects $H_0 : \theta = \theta_0$ if $\hat{\theta}_n$ is far from θ_0, calibrated by the estimated standard error.

1.3.2 Test statistic

$$W = \frac{\hat{\theta}_n - \theta_0}{\widehat{\mathrm{SE}}(\hat{\theta}_n)} \xrightarrow{d} N(0,1) \quad \text{under } H_0$$

- **Two-sided**: reject if $|W| > z_{\alpha/2}$
- **One-sided**: reject if $W > z_\alpha$ (or $W < -z_\alpha$)

This is exactly the z-test from Lecture 5, now viewed as a hypothesis test.

1.3.3 Estimating Fisher information

We need $\widehat{\mathrm{SE}}(\hat{\theta}_n)$, which requires estimating $I(\theta_0)$. Two standard approaches:

- **Option 1 (plug-in)**: $\widehat{\mathrm{SE}} = 1/\sqrt{nI(\hat{\theta}_n)}$ — replace θ_0 with $\hat{\theta}_n$ in the Fisher information formula.
- **Option 2 (observed information)**: $\widehat{\mathrm{SE}} = 1/\sqrt{-\ell_n''(\hat{\theta}_n; X)}$ — use the curvature of the log-likelihood at $\hat{\theta}_n$.

Both give consistent estimates of $1/\sqrt{nI(\theta_0)}$ — the choice doesn't matter asymptotically, but can matter in finite samples.

1.3.4 Robustness: the sandwich estimator

Both options above assume the model is correctly specified. Under misspecification, the Fisher information identity $I(\theta) = E[-\ell_1''(\theta)]$ can break down.

The **sandwich SE** estimates the variance of the score directly from data, without relying on the information identity:

$$\widehat{\mathrm{SE}}_{\text{sand}} = \frac{\sqrt{\frac{1}{n}\sum_{i=1}^{n}[\ell_1'(\hat{\theta}_n; X_i)]^2}}{I(\hat{\theta}_n)\cdot\sqrt{n}}$$

Under correct specification, sandwich $\approx$ plug-in. Under misspecification, the sandwich is **more robust** — it gives valid standard errors whether or not the model is correct, while the model-based SEs can be either too small or too large. The **bootstrap** gives yet another option (essentially equivalent to the sandwich for iid data).

1.3.5 Poisson example: three standard errors

For Pois(λ) data, the individual score is $\ell_1'(\lambda; X_i) = X_i/\lambda - 1$. At the MLE $\hat{\lambda} = \bar{X}$:

$$\ell_1'(\bar{X}; X_i) = \frac{X_i}{\bar{X}} - 1$$

The sandwich SE estimates the score variance directly:

$$\frac{1}{n}\sum_{i=1}^{n}\left[\ell_1'(\bar{X}; X_i)\right]^2 = \frac{1}{n\bar{X}^2}\sum_{i=1}^{n}(X_i - \bar{X})^2 = \frac{(n-1)s^2}{n\bar{X}^2}$$

where $s^2 = \frac{1}{n-1}\sum(X_i - \bar{X})^2$ is the sample variance. Dividing by $I(\bar{X})^2 \cdot n = n/\bar{X}^2$ and taking a square root, the sandwich SE simplifies to $s/\sqrt{n}$ — just the CLT-based SE!

Method	Formula	Derivation
Plug-in	$\sqrt{\bar{X}/n}$	Uses $I(\hat{\lambda}) = 1/\bar{X}$
Observed info	$\sqrt{\bar{X}/n}$	Uses $-\ell_n''(\hat{\lambda}) = n/\bar{X}$; same as plug-in!
Sandwich	$s/\sqrt{n}$	Estimates score variance directly; no Poisson assumption

[3]:
```
# Three standard errors for the Poisson MLE

# Option 1: Plug-in (use I(^) = 1/^ = 1/x̄)
se_plugin = np.sqrt(x_bar / n)

# Option 2: Observed information (use - ''(^) = Σx_i / ^² = n*x̄ / x̄² = n/x̄)
```

```python
se_observed = 1 / np.sqrt(n / x_bar)  # = sqrt(x̄/n), same as plug-in!

# Option 3: Sandwich (use sample variance directly - no Poisson assumption)
se_sandwich = np.std(x, ddof=1) / np.sqrt(n)

print("Three standard errors for ^ = x̄:")
print(f"  Plug-in:    {se_plugin:.4f}")
print(f"  Observed:   {se_observed:.4f}  (same as plug-in for Poisson!)")
print(f"  Sandwich:   {se_sandwich:.4f}")
print(f"\nRatio (sandwich / plug-in): {se_sandwich / se_plugin:.4f}")
print(f"\nThe sandwich SE is just ^/√n - the CLT-based SE you already know!")
print(f"It agrees with the plug-in when s²  x̄ (Poisson holds).")
print(f"Here: s² = {s2:.3f}, x̄ = {x_bar:.3f}, ratio = {s2/x_bar:.3f}")
```

```
Three standard errors for ^ = x̄:
  Plug-in:    0.3197
  Observed:   0.3197  (same as plug-in for Poisson!)
  Sandwich:   0.3422

Ratio (sandwich / plug-in): 1.0702

The sandwich SE is just ^/√n - the CLT-based SE you already know!
It agrees with the plug-in when s²  x̄ (Poisson holds).
Here: s² = 3.513, x̄ = 3.067, ratio = 1.145
```

The plug-in and observed SEs are identical here — this is special to the Poisson at the MLE, not true in general. The sandwich SE is slightly larger because the sample variance ($s^2 = 3.51$) exceeds the sample mean ($\bar{X} = 3.07$) — mild overdispersion consistent with sampling variability.

Key point: The sandwich SE is just $\hat{\sigma}/\sqrt{n}$, the CLT-based standard error from Lecture 3. It doesn't use the Poisson assumption at all. The model-based SEs exploit the Poisson relationship $\mathrm{Var}(X) = \lambda = E[X]$, but they're wrong if the model is wrong.

1.3.6 Connection to confidence intervals

By **test-CI duality** (Lecture 14), we can invert the Wald test to get a confidence interval. The Wald test rejects $H_0 : \theta = \theta_0$ at level α when:

$$\left| \frac{\hat{\theta}_n - \theta_0}{\widehat{\mathrm{SE}}} \right| > z_{\alpha/2}$$

So the set of θ_0 values **not** rejected is:

$$\left| \frac{\hat{\theta}_n - \theta_0}{\widehat{\mathrm{SE}}} \right| \leq z_{\alpha/2} \qquad \Longleftrightarrow \qquad \hat{\theta}_n - z_{\alpha/2}\,\widehat{\mathrm{SE}} \;\leq\; \theta_0 \;\leq\; \hat{\theta}_n + z_{\alpha/2}\,\widehat{\mathrm{SE}}$$

This is exactly the **asymptotic normal CI from Lecture 5**:

$$\hat{\theta}_n \pm z_{\alpha/2} \cdot \widehat{\mathrm{SE}}$$

You've been doing Wald tests all along without calling them that!

1.3.7 Advantages and disadvantages

Advantages: - Easy to compute once you have the MLE; simple to invert to give CIs (as we just saw).

Disadvantages: 1. Must compute the MLE. 2. **Depends on parameterization** — testing $\theta = \theta_0$ vs $g(\theta) = g(\theta_0)$ can give different answers. 3. Relies on **two** approximations (score $\approx$ normal AND log-likelihood $\approx$ quadratic). 4. CI can fall **outside the parameter space** — for example, the Wald CI for a Poisson rate can go negative for small $\bar{X}$!

1.4 3. The Score Test

1.4.1 Idea

Under $H_0 : \theta = \theta_0$, the score at θ_0 has mean zero: $E_{\theta_0}[\ell_n'(\theta_0; X)] = 0$. If the score is large, that's evidence against H_0.

Key advantage: we only need to evaluate the likelihood **at** θ_0 — no MLE needed!

1.4.2 Test statistic

$$Z = \frac{\ell_n'(\theta_0; X)}{\sqrt{nI(\theta_0)}} \xrightarrow{d} N(0,1) \quad \text{under } H_0$$

- **Two-sided**: reject if $|Z| > z_{\alpha/2}$
- **One-sided**: reject if $Z > z_\alpha$

This is *just* the score CLT from Lecture 4 — no quadratic approximation needed!

1.4.3 Advantages

- **No MLE required!** Only need to evaluate at θ_0.
- No need to estimate Fisher information at the MLE; we know θ_0 under H_0, so just evaluate $I(\theta_0)$.
- **Invariant to reparameterization** (unlike the Wald test).

1.4.4 Reparameterization invariance

Suppose we reparameterize: $\theta = g(\eta)$ where g is smooth with $g'(\eta) \neq 0$. By the chain rule, the score in the η-parameterization is:

$$\ell_n'^{(\eta)}(\eta_0; X) = \ell_n'^{(\theta)}(g(\eta_0); X) \cdot g'(\eta_0)$$

Since the Fisher information is the variance of the score, the score in the new parameterization differs from the old by a factor of $g'(\eta_0)$, so:

$$I^{(\eta)}(\eta_0) = \mathrm{Var}[\ell_1'^{(\eta)}(\eta_0; X)] = I^{(\theta)}(g(\eta_0)) \cdot g'(\eta_0)^2$$

So the score test statistic is:

$$\frac{\ell_n'^{(\eta)}(\eta_0; X)}{\sqrt{nI^{(\eta)}(\eta_0)}} = \frac{\ell_n'^{(\theta)}(\theta_0; X) \cdot g'(\eta_0)}{\sqrt{nI^{(\theta)}(\theta_0)} \cdot |g'(\eta_0)|} = \frac{\ell_n'^{(\theta)}(\theta_0; X)}{\sqrt{nI^{(\theta)}(\theta_0)}}$$

The g' cancels! The score test gives the **same answer regardless of parameterization**. Contrast this with the Wald test, where $(\hat{\theta}_n - \theta_0)/\widehat{\mathrm{SE}}$ changes if we reparameterize.

1.4.5 Poisson example: Wald vs Score

For Pois(λ) data, the two tests have the **same numerator but different denominators**:

Test	Statistic	Denominator evaluates SE at…
Wald	$W = \frac{\sqrt{n}(\bar{X}-\lambda_0)}{\sqrt{\bar{X}}}$	the MLE $\hat{\lambda} = \bar{X}$
Score	$Z = \frac{\sqrt{n}(\bar{X}-\lambda_0)}{\sqrt{\lambda_0}}$	the null value λ_0

1.4.6 Inverting the score test to get a CI

We can invert each to get confidence intervals. For the Wald test this is straightforward (as we just saw): $\bar{X} \pm z_{\alpha/2}\sqrt{\bar{X}/n}$.

For the **score test**, the inversion is more interesting. We want the set of λ_0 values not rejected:

$$|Z| \le z_{\alpha/2} \qquad \Longleftrightarrow \qquad \frac{n(\bar{X}-\lambda_0)^2}{\lambda_0} \le z_{\alpha/2}^2$$

Rearranging: $n(\bar{X}-\lambda_0)^2 \le z_{\alpha/2}^2\,\lambda_0$, which gives:

$$n\lambda_0^2 - (2n\bar{X} + z_{\alpha/2}^2)\lambda_0 + n\bar{X}^2 \le 0$$

Dividing through by n:

$$\lambda_0^2 - \left(2\bar{X} + \frac{z_{\alpha/2}^2}{n}\right)\lambda_0 + \bar{X}^2 \le 0$$

This is a quadratic in λ_0, so the score CI is the interval between the two roots:

$$\lambda_0 = \frac{\left(2\bar{X} + \frac{z_{\alpha/2}^2}{n}\right) \pm \sqrt{\left(2\bar{X} + \frac{z_{\alpha/2}^2}{n}\right)^2 - 4\bar{X}^2}}{2}$$

Notice that both roots are **always positive** (since all the terms are positive), so the score CI never goes below zero — unlike the Wald CI, which can go negative for small $\bar{X}$. The score CI respects the natural constraint $\lambda > 0$.

[4]:
```
# Figure 1: Wald vs Score vs GLRT confidence intervals for Poisson
z_crit = norm.ppf(0.975)
chi2_crit = chi2.ppf(0.95, df=1)

# --- Wald CI ---
wald_lo = x_bar - z_crit * np.sqrt(x_bar / n)
wald_hi = x_bar + z_crit * np.sqrt(x_bar / n)

# --- Score CI ---
# Solve n(x̄ -  )² /  = z² => n ² - (2nx̄ + z²)  + nx̄² = 0
a_coef = n
b_coef = -(2 * n * x_bar + z_crit**2)
c_coef = n * x_bar**2
disc = b_coef**2 - 4 * a_coef * c_coef
score_lo = (-b_coef - np.sqrt(disc)) / (2 * a_coef)
score_hi = (-b_coef + np.sqrt(disc)) / (2 * a_coef)

# --- GLRT CI ---
# Solve 2n[x̄ log(x̄/ ) - (x̄ -  )] =  ²_{1, }
def glrt_stat(lam):
    if lam <= 0:
        return np.inf
    return 2 * n * (x_bar * np.log(x_bar / lam) - (x_bar - lam))

glrt_lo = brentq(lambda lam: glrt_stat(lam) - chi2_crit, 0.01, x_bar)
glrt_hi = brentq(lambda lam: glrt_stat(lam) - chi2_crit, x_bar, x_bar * 5)

# --- Plot ---
fig, ax = plt.subplots(figsize=(10, 3.5))

methods = ['Wald', 'Score', 'GLRT']
lowers = [wald_lo, score_lo, glrt_lo]
uppers = [wald_hi, score_hi, glrt_hi]
colors = [COLOR_DATA, COLOR_APPROX, '#666666']
y_positions = [2, 1, 0]

for method, lo, hi, color, y in zip(methods, lowers, uppers, colors,␣
 ↪y_positions):
    ax.plot([lo, hi], [y, y], color=color, linewidth=3, solid_capstyle='round')
    ax.plot(x_bar, y, 'o', color=color, markersize=8, zorder=5)
    ax.text(lo - 0.05, y + 0.25, f'[{lo:.3f}, {hi:.3f}]',
            ha='right', va='bottom', fontsize=9.5, color=color,␣
 ↪fontweight='bold')
```

```
ax.axvline(lam_true, color='black', linestyle='--', linewidth=1.5,
           label=f'True $\\lambda = {lam_true}$')
ax.axvline(0, color='gray', linestyle=':', linewidth=1, alpha=0.5)

ax.set_xlabel('$\\lambda$', fontsize=12)
ax.set_yticks(y_positions)
ax.set_yticklabels(methods, fontsize=11, fontweight='bold')
ax.set_title(f'95% confidence intervals for $\\lambda$ ($n = {n}$, $\\bar{{X}}␣
↪= {x_bar:.3f}$)',
             fontsize=13, fontweight='bold')
ax.legend(fontsize=10, loc='upper right')
ax.set_xlim(-0.3, 5.5)
ax.set_ylim(-0.7, 3.0)
plt.tight_layout()
plt.show()
```

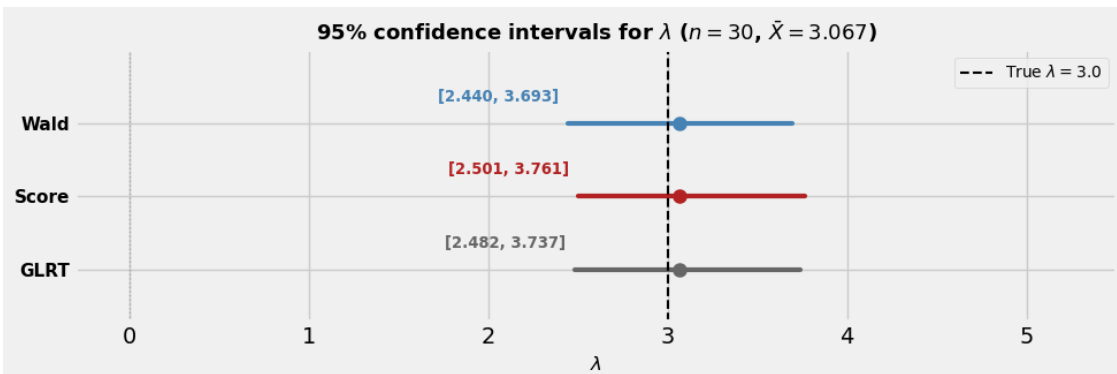


Figure 1. *Three 95% confidence intervals for the Poisson rate λ from the same data ($n = 30$, $\bar{X} = 3.07$). The* ***Wald CI*** *(blue) is centered on $\bar{X}$ and symmetric. The* ***Score CI*** *(red) and* ***GLRT CI*** *(gray) are slightly asymmetric and always contained in $(0, \infty)$. For this sample, all three are similar, but they can differ substantially when $\bar{X}$ is small — the Wald CI can go negative, while the others cannot. The dashed line marks the true $\lambda = 3$, which all three CIs cover.*

1.5 4. The Generalized Likelihood Ratio Test

1.5.1 Idea

Compare the maximum log-likelihood (at $\hat{\theta}_n$) to the log-likelihood at the null value θ_0. If H_0 is true, these shouldn't be too different.

1.5.2 Test statistic

$$\text{GLRT statistic} = 2[\ell_n(\hat{\theta}_n; X) - \ell_n(\theta_0; X)]$$

This is always ≥ 0 (since $\hat{\theta}_n$ maximizes ℓ_n). Reject H_0 if it is large.

1.5.3 Derivation of the null distribution

Taylor-expand $\ell_n(\theta)$ around $\hat{\theta}_n$, using the fact that $\ell_n'(\hat{\theta}_n) = 0$:

$$\ell_n(\theta_0; X) - \ell_n(\hat{\theta}_n; X) \approx \frac{1}{2}\ell_n''(\hat{\theta}_n; X)(\theta_0 - \hat{\theta}_n)^2$$

Since $-\ell_n''(\hat{\theta}_n; X) \approx nI(\theta_0)$ by the law of large numbers, we get:

$$2[\ell_n(\hat{\theta}_n; X) - \ell_n(\theta_0; X)] \approx nI(\theta_0)(\hat{\theta}_n - \theta_0)^2$$

Under H_0: $\sqrt{nI(\theta_0)}(\hat{\theta}_n - \theta_0) \xrightarrow{d} N(0,1)$, so squaring:

$$2[\ell_n(\hat{\theta}_n; X) - \ell_n(\theta_0; X)] \xrightarrow{d} \chi_1^2$$

Reject H_0 if $2[\ell_n(\hat{\theta}_n; X) - \ell_n(\theta_0; X)] \geq \chi_{1,\alpha}^2$.

1.5.4 Poisson example

For Pois(λ) data with log-likelihood $\ell_n(\lambda) = n\bar{X}\log\lambda - n\lambda + C$, the GLRT statistic is:

$$2[\ell_n(\hat{\lambda}) - \ell_n(\lambda_0)] = 2[n\bar{X}\log\bar{X} - n\bar{X} - n\bar{X}\log\lambda_0 + n\lambda_0] = 2n\left[\bar{X}\log\frac{\bar{X}}{\lambda_0} - (\bar{X} - \lambda_0)\right]$$

Note that each term in the brackets is non-negative: $\bar{X}\log(\bar{X}/\lambda_0) \geq \bar{X} - \lambda_0$ by Jensen's inequality (with equality iff $\bar{X} = \lambda_0$).

This involves a logarithm — more complex than the Wald or Score statistics, but uses **no Fisher information**. The GLRT CI (the set of λ_0 values not rejected) is always positive but must be solved numerically, since the equation $\bar{X}\log(\bar{X}/\lambda_0) - (\bar{X} - \lambda_0) = c$ has no closed-form solution for λ_0.

Note the GLRT is **inherently two-sided** since the statistic is non-negative. One-sided tests require the Wald or score approach.

1.5.5 Advantages and disadvantages

Advantages: - Only requires computing the MLE and evaluating ℓ_n at two points; **no Fisher information** needed. - Generalizes naturally to composite-vs-composite settings (Lecture 16).

Disadvantages: - Must compute the MLE. - Inherently two-sided in this form. - CI must be solved numerically in general.

1.6 5. The Unifying Picture: Three Measurements of One Parabola

1.6.1 The quadratic approximation diagram

Here is the key idea of this lecture. The log-likelihood $\ell_n(\theta)$ is approximately a **downward parabola** near the MLE $\hat{\theta}_n$. The three tests read off different features of this parabola:

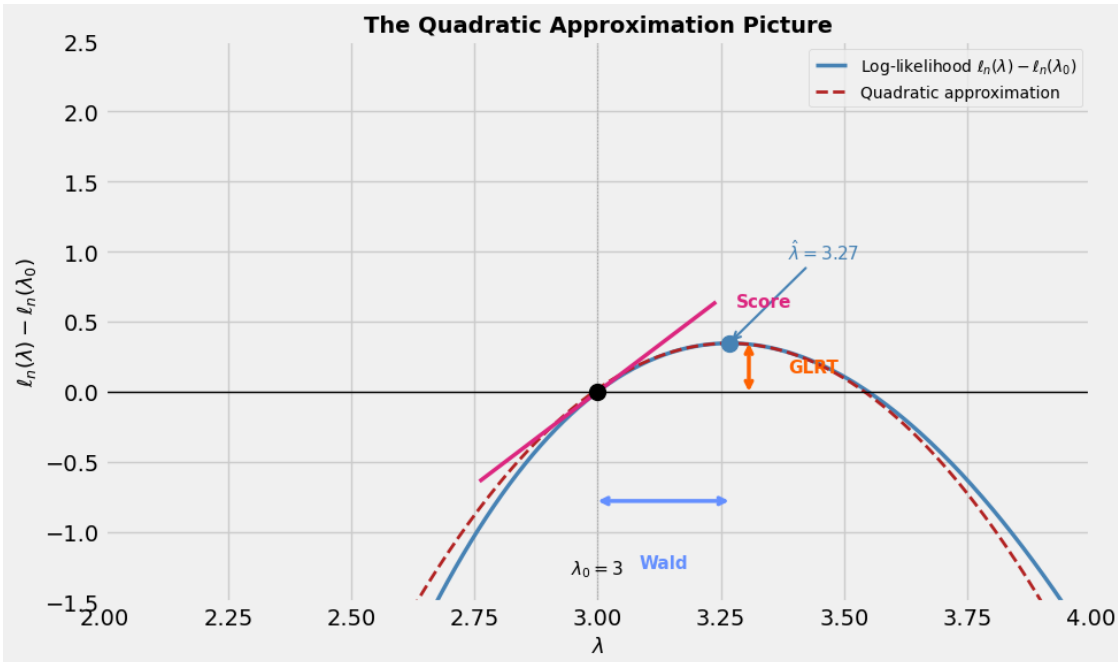
- **Wald**: the **horizontal distance** from θ_0 to $\hat{\theta}_n$ (scaled by curvature)
- **Score**: the **slope** of ℓ_n at θ_0 (the tangent line)
- **GLRT**: the **vertical drop** from $\ell_n(\hat{\theta}_n)$ to $\ell_n(\theta_0)$

If the log-likelihood were exactly a parabola, all three would give identical test statistics. They differ only because the actual log-likelihood is not perfectly quadratic.

[5]:
```
# Figure 2: The quadratic approximation picture
# RE-RUN THIS CELL to generate a new random sample and see how the
# likelihood curve (and all three test statistics) change.
# The grid stays fixed - only the curve moves.

lam_0 = 3.0    # Fixed null value
lam_true_viz = lam_0  # Simulate under H
n_viz = 30

# Fixed axis limits (don't change with data)
xlims = (2, 4)
ylims = (-1.5, 2.5)

# Fresh random data each time this cell is run
x_viz = np.random.default_rng().poisson(lam_true_viz, n_viz)
x_bar_viz = x_viz.mean()

# Log-likelihood relative to  ( ): the curve passes through ( , 0)
lam_grid = np.linspace(xlims[0] * 0.9, xlims[1] * 1.1, 500)
ll_relative = n_viz * (x_bar_viz * np.log(lam_grid / lam_0) - (lam_grid -␣
↪lam_0))

# Quadratic approximation (parabola centered at x̄, peak = GLRT/2)
glrt_half = n_viz * (x_bar_viz * np.log(x_bar_viz / lam_0) - (x_bar_viz -␣
↪lam_0))
ll_quad = glrt_half - n_viz / (2 * x_bar_viz) * (lam_grid - x_bar_viz)**2

# Score at   (slope of the curve at the origin)
score_at_lam0 = n_viz * (x_bar_viz / lam_0 - 1)

# Test statistics
W = np.sqrt(n_viz) * (x_bar_viz - lam_0) / np.sqrt(x_bar_viz)
Z = np.sqrt(n_viz) * (x_bar_viz - lam_0) / np.sqrt(lam_0)
glrt_val = 2 * glrt_half
```

```python
# Adaptive sizes based on axis range
dx = xlims[1] - xlims[0]
dy = ylims[1] - ylims[0]

# Arrow style: small heads, minimal padding
arrow_kw = dict(lw=2.5, shrinkA=0, shrinkB=0, mutation_scale=10)

# Tangent line at ( , 0) - scaled to axis range
tangent_hw = 0.12 * dx
tangent_x = np.linspace(lam_0 - tangent_hw, lam_0 + tangent_hw, 50)
tangent_y = score_at_lam0 * (tangent_x - lam_0)

fig, ax = plt.subplots(figsize=(10, 6))

# Curves
ax.plot(lam_grid, ll_relative, color=COLOR_DATA, linewidth=2.5,
        label=r'Log-likelihood $\ell_n(\lambda) - \ell_n(\lambda_0)$')
ax.plot(lam_grid, ll_quad, color=COLOR_APPROX, linewidth=2, linestyle='--',
        label='Quadratic approximation')

# Mark MLE (peak) and null (origin)
ax.plot(x_bar_viz, glrt_half, 'o', color=COLOR_DATA, markersize=10, zorder=5)
ax.plot(lam_0, 0, 'o', color='black', markersize=10, zorder=5)

# Annotate MLE - place label adaptively
mle_label_x = x_bar_viz + 0.06 * dx
mle_label_y = min(glrt_half + 0.15 * dy, ylims[1] - 0.1 * dy)
ax.annotate(f'$\\hat\\lambda = {x_bar_viz:.2f}$', xy=(x_bar_viz, glrt_half),
            xytext=(mle_label_x, mle_label_y), fontsize=11,
            fontweight='bold', color=COLOR_DATA,
            arrowprops=dict(arrowstyle='->', color=COLOR_DATA, lw=1.5))

# === Three test annotations ===

# WALD: horizontal arrow from   to x̄
y_wald = ylims[0] + 0.18 * dy
ax.annotate('', xy=(x_bar_viz, y_wald), xytext=(lam_0, y_wald),
            arrowprops=dict(arrowstyle='<->', color='#648FFF', **arrow_kw))
ax.text((x_bar_viz + lam_0) / 2, y_wald - 0.12 * dy,
        'Wald', ha='center', fontsize=11, color='#648FFF', fontweight='bold')

# SCORE: tangent line at ( , 0)
ax.plot(tangent_x, tangent_y, color='#DC267F', linewidth=2.5, zorder=4)
score_label_x = lam_0 + tangent_hw + 0.02 * dx
score_label_y = tangent_y[-1]
ax.text(score_label_x, score_label_y,
        'Score', fontsize=11, color='#DC267F', fontweight='bold', va='center')
```

```python
# GLRT: vertical arrow from y=0 to peak at x̄ (always shown)
x_glrt = x_bar_viz + 0.02 * dx
ax.annotate('', xy=(x_glrt, glrt_half), xytext=(x_glrt, 0),
            arrowprops=dict(arrowstyle='<->', color='#FE6100', **arrow_kw))
ax.text(x_glrt + 0.04 * dx, glrt_half / 2,
        'GLRT', fontsize=11, color='#FE6100', fontweight='bold', va='center')

# Reference lines and label
ax.axhline(0, color='black', linewidth=1.0)
ax.axvline(lam_0, color='gray', linewidth=0.8, linestyle=':', alpha=0.5)
ax.text(lam_0, ylims[0] + 0.05 * dy, f'$\\lambda_0 = {lam_0:.0f}$',
        ha='center', fontsize=11, fontweight='bold')

ax.set_xlabel('$\\lambda$', fontsize=13)
ax.set_ylabel('$\\ell_n(\\lambda) - \\ell_n(\\lambda_0)$', fontsize=13)
ax.set_title('The Quadratic Approximation Picture', fontsize=14,␣
 ↪fontweight='bold')
ax.legend(fontsize=10, loc='upper right')

# FIXED axis limits - grid stays the same when you re-run
ax.set_xlim(xlims)
ax.set_ylim(ylims)
plt.tight_layout()
plt.show()

# Print summary
print(f"x̄ = {x_bar_viz:.3f}  (MLE)")
print(f"Score at  :  '_n( ) = {score_at_lam0:.2f}")
print(f"  Wald:  W = {W:+.3f}   (p = {2*(1-norm.cdf(abs(W))):.4f})")
print(f"  Score: Z = {Z:+.3f}   (p = {2*(1-norm.cdf(abs(Z))):.4f})")
print(f"  GLRT:    = {glrt_val:.3f}   (p = {1-chi2.cdf(glrt_val, 1):.4f})")
```




```
x̄ = 3.267  (MLE)
Score at  :  '_n( ) = 2.67
  Wald:  W = +0.808   (p = 0.4190)
  Score: Z = +0.843   (p = 0.3991)
  GLRT:    = 0.691   (p = 0.4058)
```

Figure 2. *The unifying picture for the three asymptotic tests, with data simulated under H_0 ($\lambda = \lambda_0 = 3$). The* ***log-likelihood*** *(blue solid) is plotted relative to its value at the null, so the curve passes through $(\lambda_0, 0)$ and peaks at the MLE $\hat{\lambda} = \bar{X}$. The three tests measure different features of this curve: the* ***Wald test*** *(blue arrow) measures the horizontal distance from λ_0 to $\hat{\lambda}$; the* ***score test*** *(magenta tangent line) measures the slope at λ_0; the* ***GLRT*** *(orange arrow) measures the peak height above 0. The score at λ_0 determines both shifts: a larger score means $\hat{\lambda}$ is further from λ_0 (horizontal) and the peak is higher (vertical).* ***Re-run this cell*** *to generate a new random sample and see the curve change while the grid stays fixed — most of the time H_0 is not rejected, but occasionally the random score is large enough to produce a significant test.*

1.6.2 Asymptotic equivalence

For large n, the quadratic approximation becomes increasingly accurate. In particular, under H_0:

- $\ell_n(\hat{\theta}_n) - \ell_n(\theta_0) \approx \frac{1}{2} n I(\theta_0)(\hat{\theta}_n - \theta_0)^2$ (GLRT $\leftrightarrow$ Wald)
- $\hat{\theta}_n - \theta_0 \approx \frac{\ell_n'(\theta_0; X)}{n I(\theta_0)}$ (Wald $\leftrightarrow$ Score)

So: $W^2 \approx Z^2 \approx 2[\ell_n(\hat{\theta}_n) - \ell_n(\theta_0)]$, all $\xrightarrow{d} \chi^2_1$.

The signed Wald (W) and score (Z) statistics are both $\xrightarrow{d} N(0, 1)$.

They can differ in finite samples, and there is no general ordering of their power.

1.6.3 When to use which?

- **Wald**: When you've already computed the MLE (the default in most software); gives CIs for free.
- **Score**: When computing the MLE is hard or when you want invariance to reparameterization; only need θ_0.
- **GLRT**: When you can compute the MLE and evaluate ℓ_n at θ_0; most natural for comparing nested models (Lecture 16).
- In practice, all three give similar results for large samples; the choice is often one of convenience.

[6]:
```
# Figure 3 (interactive): Vary   and see the three test statistics update

def plot_quadratic_interactive(lam_0=2.0):
    lam_grid = np.linspace(0.5, 6.5, 500)
    ll_rel = n * (x_bar * np.log(lam_grid / x_bar) - (lam_grid - x_bar))
    ll_quad_i = -n / (2 * x_bar) * (lam_grid - x_bar)**2

    # Values at
    ll_at_l0 = n * (x_bar * np.log(lam_0 / x_bar) - (lam_0 - x_bar))
    score_at_l0 = n * (x_bar / lam_0 - 1)

    # Three test statistics
    W = np.sqrt(n) * (x_bar - lam_0) / np.sqrt(x_bar)
    Z = np.sqrt(n) * (x_bar - lam_0) / np.sqrt(lam_0)
    if lam_0 > 0 and x_bar > 0:
        glrt_val = 2 * n * (x_bar * np.log(x_bar / lam_0) - (x_bar - lam_0))
    else:
        glrt_val = np.nan

    # Tangent line
    t_x = np.linspace(lam_0 - 0.5, lam_0 + 0.5, 50)
    t_y = ll_at_l0 + score_at_l0 * (t_x - lam_0)

    fig, axes = plt.subplots(1, 2, figsize=(14, 5.5),
                             gridspec_kw={'width_ratios': [2.5, 1]})

    # Left: log-likelihood picture
    ax = axes[0]
    ax.plot(lam_grid, ll_rel, color=COLOR_DATA, linewidth=2.5,
            label='Log-likelihood')
    ax.plot(lam_grid, ll_quad_i, color=COLOR_APPROX, linewidth=2,
            linestyle='--', label='Quadratic approx.')

    ax.plot(x_bar, 0, 'o', color=COLOR_DATA, markersize=9, zorder=5)
    ax.plot(lam_0, ll_at_l0, 'o', color='black', markersize=8, zorder=5)

    # Wald arrow
```

```
y_w = -1.0
ax.annotate('', xy=(x_bar, y_w), xytext=(lam_0, y_w),
            arrowprops=dict(arrowstyle='<->', color='#648FFF', lw=2.5))

# Score tangent
ax.plot(t_x, t_y, color='#DC267F', linewidth=2)

# GLRT arrow
if ll_at_l0 < -0.5:
    offset = -0.12 if lam_0 < x_bar else 0.12
    ax.annotate('', xy=(lam_0 + offset, 0),
                xytext=(lam_0 + offset, ll_at_l0),
                arrowprops=dict(arrowstyle='<->', color='#FE6100', lw=2.5))

ax.axhline(0, color='gray', linewidth=0.8)
ax.set_xlabel('$\\lambda$', fontsize=12)
ax.set_ylabel('$\\ell_n(\\lambda) - \\ell_n(\\hat\\lambda)$', fontsize=12)
ax.set_title(f'$\\lambda_0 = {lam_0:.2f}$, $\\hat\\lambda = {x_bar:.2f}$',
             fontsize=13, fontweight='bold')
ax.legend(fontsize=9, loc='upper right')
ax.set_xlim(0.5, 6.5)
ax.set_ylim(-25, 5)

# Right: test statistics table
ax2 = axes[1]
ax2.axis('off')
table_data = [
    ['Test', 'Statistic', '$p$-value'],
    ['Wald ($W$)', f'{W:.3f}', f'{2*(1-norm.cdf(abs(W))):.4f}'],
    ['Score ($Z$)', f'{Z:.3f}', f'{2*(1-norm.cdf(abs(Z))):.4f}'],
    ['GLRT', f'{glrt_val:.3f}',
     f'{1-chi2.cdf(glrt_val, 1):.4f}' if not np.isnan(glrt_val) else 'N/A'],
]
table = ax2.table(cellText=table_data[1:], colLabels=table_data[0],
                  loc='center', cellLoc='center')
table.auto_set_font_size(False)
table.set_fontsize(11)
table.scale(1, 1.8)
for i, color in enumerate(['#648FFF', '#DC267F', '#FE6100']):
    for j in range(3):
        table[(i+1, j)].set_text_props(color=color, fontweight='bold')
ax2.set_title('Test statistics (two-sided)', fontsize=12, fontweight='bold',
              pad=20)

plt.tight_layout()
plt.show()
```

```
lam0_slider = widgets.FloatSlider(
    value=2.0, min=0.5, max=6.0, step=0.1,
    description='Null $\\lambda_0$:',
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='500px')
)

widgets.interact(plot_quadratic_interactive, lam_0=lam0_slider);
```

```
interactive(children=(FloatSlider(value=2.0, description='Null $\\lambda_0$:',␣
↪layout=Layout(width='500px'), m…
```

Figure 3 (interactive). *Move the slider to change λ_0 and see how the three tests respond.* ***Left:*** *the log-likelihood curve (blue) and its quadratic approximation (red dashed), with the Wald (blue ↔), Score (magenta tangent), and GLRT (orange ↕) highlighted.* ***Right:*** *the resulting test statistics and two-sided p-values. When λ_0 is near $\hat{\lambda} = \bar{X}$, all three tests agree closely and give large p-values. As λ_0 moves away, the tests increasingly disagree because the log-likelihood becomes less quadratic.*

1.6.4 Example: Normal mean

For $X_1, \ldots, X_n \sim N(\mu, \sigma^2)$ with σ^2 known, the log-likelihood is:

$$\ell_n(\mu) = -\frac{n}{2\sigma^2}(\mu - \bar{X})^2 + C$$

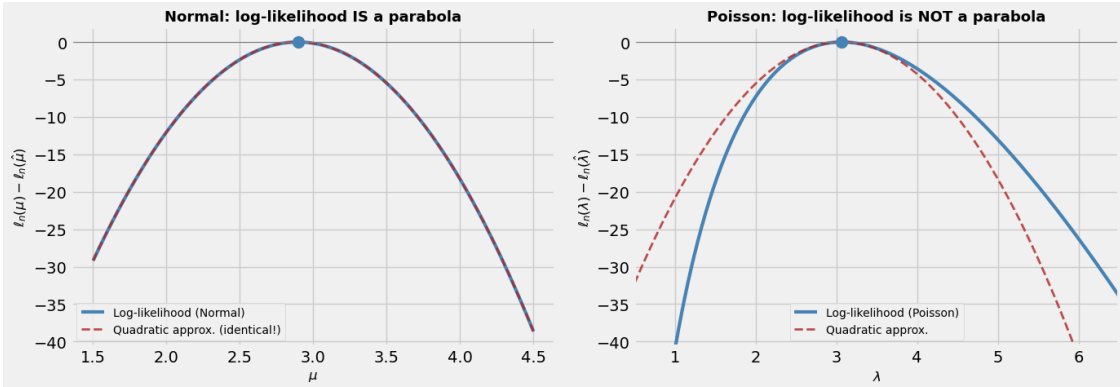
This is **exactly a parabola** — so the quadratic approximation is perfect, and all three tests coincide:

Test	Statistic
Wald	$W = \sqrt{n}(\bar{X} - \mu_0)/\sigma$
Score	$Z = \sqrt{n}(\bar{X} - \mu_0)/\sigma$ (same!)
GLRT	$2[\ell_n(\bar{X}) - \ell_n(\mu_0)] = n(\bar{X} - \mu_0)^2/\sigma^2 = W^2 = Z^2$

Why do all three agree exactly? Because the log-likelihood is exactly a parabola — there is no approximation error.

[7]:
```
# Figure 4: Normal example - the log-likelihood IS the quadratic approximation
sigma = 1.0
n_norm = 30
x_norm = rng.normal(loc=3.0, scale=sigma, size=n_norm)
x_bar_norm = x_norm.mean()
mu_0 = 2.5

mu_grid = np.linspace(1.5, 4.5, 500)
ll_norm = -n_norm / (2 * sigma**2) * (mu_grid - x_bar_norm)**2
ll_quad_norm = ll_norm.copy()  # identical!

# Test statistics
```

```
W_norm = np.sqrt(n_norm) * (x_bar_norm - mu_0) / sigma
Z_norm = np.sqrt(n_norm) * (x_bar_norm - mu_0) / sigma  # same!
glrt_norm = n_norm * (x_bar_norm - mu_0)**2 / sigma**2   # = W² = Z²

fig, axes = plt.subplots(1, 2, figsize=(14, 5),
                         gridspec_kw={'width_ratios': [1, 1]})

# Left: Normal log-likelihood = exact parabola
ax = axes[0]
ax.plot(mu_grid, ll_norm, color=COLOR_DATA, linewidth=3,
        label='Log-likelihood (Normal)')
ax.plot(mu_grid, ll_quad_norm, color=COLOR_APPROX, linewidth=2,
        linestyle='--', alpha=0.8, label='Quadratic approx. (identical!)')
ax.plot(x_bar_norm, 0, 'o', color=COLOR_DATA, markersize=10, zorder=5)
ax.axhline(0, color='gray', linewidth=0.8)
ax.set_xlabel('$\\mu$', fontsize=12)
ax.set_ylabel('$\\ell_n(\\mu) - \\ell_n(\\hat\\mu)$', fontsize=12)
ax.set_title('Normal: log-likelihood IS a parabola', fontsize=13,
             fontweight='bold')
ax.legend(fontsize=10)

# Right: Compare to Poisson
ax2 = axes[1]
lam_grid_p = np.linspace(0.5, 6.5, 500)
ll_pois = n * (x_bar * np.log(lam_grid_p / x_bar) - (lam_grid_p - x_bar))
ll_quad_p = -n / (2 * x_bar) * (lam_grid_p - x_bar)**2
ax2.plot(lam_grid_p, ll_pois, color=COLOR_DATA, linewidth=3,
         label='Log-likelihood (Poisson)')
ax2.plot(lam_grid_p, ll_quad_p, color=COLOR_APPROX, linewidth=2,
         linestyle='--', alpha=0.8, label='Quadratic approx.')
ax2.plot(x_bar, 0, 'o', color=COLOR_DATA, markersize=10, zorder=5)
ax2.axhline(0, color='gray', linewidth=0.8)
ax2.set_xlabel('$\\lambda$', fontsize=12)
ax2.set_ylabel('$\\ell_n(\\lambda) - \\ell_n(\\hat\\lambda)$', fontsize=12)
ax2.set_title('Poisson: log-likelihood is NOT a parabola', fontsize=13,
              fontweight='bold')
ax2.legend(fontsize=10)
ax2.set_xlim(0.5, 6.5)
ax2.set_ylim(axes[0].get_ylim())

plt.tight_layout()
plt.show()

print(f"Normal: W = {W_norm:.4f}, Z = {Z_norm:.4f}, "
      f"\u221AGLRT = {np.sqrt(glrt_norm):.4f}  \u2190 all identical!")
```




```
Normal: W = 2.1638, Z = 2.1638, √GLRT = 2.1638  ← all identical!
```

Figure 4. *Why all three tests agree for the Normal but not the Poisson.* ***Left:*** *for $N(\mu, \sigma^2)$ data, the log-likelihood is exactly a downward parabola — the red dashed curve sits perfectly on the blue solid curve. The Wald, Score, and GLRT give identical test statistics.* ***Right:*** *for Pois(λ) data, the log-likelihood curves more steeply to the left (toward $\lambda = 0$) than to the right. The quadratic approximation is good near the MLE but imperfect further away, which is why the three tests disagree.*

1.7 6. Summary and Midterm Preview

1.7.1 Today

Three asymptotic tests for $H_0 : \theta = \theta_0$ in a one-parameter family, all based on the quadratic approximation to the log-likelihood:

Test	Statistic	What it measures	Needs MLE?	Needs $I(\theta)$?
Wald	$W = \frac{\hat{\theta}-\theta_0}{\widehat{\mathrm{SE}}}$	Horizontal distance	Yes	Yes (at $\hat{\theta}$)
Score	$Z = \frac{\ell_n'(\theta_0)}{\sqrt{nI(\theta_0)}}$	Slope at θ_0	**No**	Yes (at θ_0)
GLRT	$2[\ell_n(\hat{\theta}) - \ell_n(\theta_0)]$	Vertical drop	Yes	**No**

All three are asymptotically $N(0, 1)$ (signed) or χ^2_1 (squared) under H_0. They are equivalent when the quadratic approximation is exact (Normal), and differ only due to non-quadratic features of the log-likelihood.

1.7.2 Key takeaways

1. The **quadratic approximation picture** unifies all three: they measure horizontal, tangential, and vertical features of the same log-likelihood parabola.
2. The **score test** only requires evaluating at θ_0 (no MLE needed) and is invariant to reparameterization.

3. The **Wald test** gives CIs for free via test-CI duality.
4. The **GLRT** generalizes naturally to more complex settings (Lecture 16).

1.7.3 Coming up

- **Lecture 16**: The GLRT generalizes to composite-vs-composite testing, leading to Wilks' theorem, the χ^2 goodness-of-fit test, and Pearson's χ^2 test for categorical data.

1.7.4 Midterm preview

- The midterm exam is **Friday, March 13** (evening).
- Coverage: **Lectures 1–14**, Worksheets 1–7 (today's material is **not** on the midterm).
- Key themes: convergence in probability and distribution (Lectures 2–3), MLE and Fisher information (Lectures 3–5), decision theory and estimation (Lecture 6), Bayesian inference (Lectures 7–8), bootstrap (Lectures 9–10), hypothesis testing framework (Lectures 12–14).